Question 039 | Question bank page 4 | Q.1

Write a C# program to demonstrate Multiple Inheritance using Interfaces.

Project type: C# Console Application

Steps to perform

1. Open Visual Studio and choose Create a new project.

2. Select Console App (C#), choose a project name and create it.

3. Open Program.cs and replace the sample code with the provided C# code.

4. Build the project; resolve any missing using directives or compilation errors.

5. Press Ctrl+F5, enter the required inputs, and verify the console output.

Closest supplied practical: } } Write the program with Inheritance (all types)

Source: Dot_Net_Practicals_1_to_5_.pdf

Provided creation instructions and source files

```
Creation steps: Create the Console App named Floydtriangle. Add the listed files at the locations
below and enter the
corresponding code.
Filename: Program.cs    Location: Project root
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace InheritanceDemo
{
    class Animal
    {
        public void Eat() => Console.WriteLine("Animal is eating");
    }
    class Dog : Animal
    {
        public void Bark() => Console.WriteLine("Dog is barking");
    }
    class GrandParent
    {
        public void Method1() => Console.WriteLine("GrandParent method");
    }
    class Parent : GrandParent
    {
        public void Method2() => Console.WriteLine("Parent method");
    }
    class Child : Parent
    {
        public void Method3() => Console.WriteLine("Child method");
    }
    class Shape
    {
        public void Draw() => Console.WriteLine("Drawing a shape");
    }
    class Circle : Shape
    {
        public void Info() => Console.WriteLine("This is a circle");
    }
    class Rectangle : Shape
    {
        public void Info() => Console.WriteLine("This is a rectangle");
    }
    interface IFlyable
    {
        void Fly();
    }
    interface ISwimable
    {
        void Swim();
    }
    class Duck : IFlyable, ISwimable
    {
        public void Fly() => Console.WriteLine("Duck can fly");
        public void Swim() => Console.WriteLine("Duck can swim");
    }
```

```
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("---Single Inheritance---");
            Dog d = new Dog();
            d.Eat(); d.Bark();
            Console.WriteLine("---Multilevel Inheritance---");
            Child c = new Child();
            c.Method1(); c.Method2(); c.Method3();
            Console.WriteLine("---Hierarchical Inheritance---");
            Circle cir = new Circle(); cir.Draw(); cir.Info();
            Rectangle rec = new Rectangle(); rec.Draw(); rec.Info();
            Console.WriteLine("---Multiple Inheritance(interface)---");
            Duck duck = new Duck();
            duck.Fly(); duck.Swim();
            Console.ReadKey();
        }
    }
}
```

CHECK: This is a related source example, not necessarily a complete or exact solution to the wording above. Adapt and verify it.

Expected result / verification

The application should meet the exact question requirements. Record actual output or screenshots after running; no execution has been claimed in this manual.